Onyx — Developer Guide

Onyx · multi-process OS on Raspberry Pi 4

This guide explains how to **build** Onyx, how to **write an application** or a `/bin` **tool**, how to **extend the** `kapi` **ABI**, and the **conventions** + **pitfalls** to be aware of. For the details of how things work internally, see Kernel internals.

Contents

---

1. Prerequisites and toolchain

- **AArch64 bare-metal toolchain**: `aarch64-none-elf-` (GCC), used under **WSL** on a Windows development machine.
- **Circle**, pulled in as a **git submodule** at `circle/` (our fork `stephaneweg/circle`, branch `onyx`).
- `make`, `cp`, `mkdir` (standard Unix tools, via WSL).
- A **FAT32 SD card** and a **Raspberry Pi 4** to run on (there is no QEMU `raspi4b` in the reference dev environment; bring-up is done directly on hardware).

> **Important — which Circle?** `circle/` is a **git submodule** → our fork `stephaneweg/circle` (branch `onyx`), **not** any other clone. Patch Circle *there* (commit on `onyx`), rebuild the affected library, then record the new commit in the superproject (`git add circle && git commit`). The patches are listed in Circle Changes.

2. Building Circle (once)

```
# Circle is a git submodule (the fork stephaneweg/circle, branch onyx). Fetch it
# (or clone Onyx with --recurse-submodules in the first place):
git submodule update --init --recursive

# Configure for RPi 4 / AArch64. DEPTH=32 is REQUIRED: our GImage renders 32-bit
# pixels (Circle defaults to 16).
cd circle && ./configure -r 4 -p aarch64-none-elf- -d DEPTH=32 -f
```

```
# Build the libraries used by the kernel:
(cd lib && make -j4) \
  && (cd lib/sched && make -j4) \
  && (cd lib/fs && make -j4 && cd fat && make -j4) \
  && (cd lib/usb && make -j4) \
  && (cd lib/input && make -j4) \
  && (cd addon/SDCard && make -j4) \
  && (cd addon/fatfs && make -j4)
cd ..
```

> If you change `DEPTH` later, run `make clean` in `circle/lib` before rebuilding (the `.o` files do not depend on `Config.mk`).

The libraries linked by the kernel (cf. `kernel/Makefile`): `libsched.a`, `libfatfs.a`, `libsdcard.a`, `libfs.a`, `libusb.a`, `libinput.a`, `libcircle.a`.

> The Onyx-specific patches carried by this fork (branch `onyx`, on upstream tag `Step51`) are documented in Circle Changes.

3. Building the kernel and applications

From `kernel/`:

```
cd kernel
make            # -> kernel8-rpi4.img, THEN builds the apps (../user)
```

The default target (`all`):

1. compiles our sources + links against the Circle libs → **`kernel8-rpi4.img`**;
2. triggers `make -C ../user`, which builds **all the apps** (`user/*.elf`) and the **`/bin` tools** (`user/bin/*.elf`).

> **Kernel → apps order.** Since the apps go through the **fixed-address ABI table** and are **not** linked against the kernel's addresses, they no longer depend on the kernel build: a *kernel-only* change (that respects the ABI) does **not** require recompiling the apps. `make` rebuilds them anyway for convenience.

Details of the kernel wiring (which Circle files we replace, the order of the `-I compat -I include` include path before `circle/include`): see Kernel internals §1 and `kernel/Makefile`.

4. Deploying to the SD card

```
cd kernel
make stage      # copie l'image + chaque app + chaque outil vers ../sdcard/
```

`make stage`:

- copies `kernel8-rpi4.img` → `sdcard/`;
- for each `../user/<name>.elf`: creates `sdcard/apps/<name>.app/` and copies the ELF into it as **`main.elf`** (the "Onyx" layout);
- copies each `../user/bin/<tool>.elf` → `sdcard/bin/<tool>.elf`.

Then copy **all** of the contents of `sdcard/` onto a **FAT32** card and boot the Pi 4. See the user guide for details about the card and the configuration files.

Keyboard layouts live as binary files in `sdcard/etc/keymaps/<NAME>.kmap` (the `keyb` tool — and the theme editor's dropdown, which lists whatever `.kmap` files are actually present — load one and send it to the kernel via `kapi_set_keymap_data`, v27). **The kernel compiles in no keyboard map at all**: at boot the keyboard table is empty and stays so until `keyb` loads a layout from the card (see Circle Changes §1), so a layout is *only* ever a file — adding one needs no kernel rebuild. Regenerate the `.kmap` files with `python tools/keymaps/genkeymaps.py`; it compiles each `<NAME>` from a `keymap_<name>.h` table in **`tools/keymaps/maps/`** (the 8 layout sources, incl. `BE` = Belgian azerty, all tracked in the Onyx repo — not in the Circle tree). The `.kmap` format is `"OKM1"` + `u16` rows/cols + the `u16[128][5]` table (see the script header).

5. The application model

A Onyx application is a **single `.c` file** compiled into a **freestanding ELF** and run in **EL1** in its own page table.

Runtime (`user/crt0.S`):

```
_start:
    stp x29, x30, [sp, #-16]!   /* the stack is already set up by the kernel */
    bl  main                    /* call main() */
    ldp x29, x30, [sp], #16
    ret                         /* return -> the kernel terminates the task */
```

No argv is passed via the stack: `main()` takes no arguments. An app retrieves its argument line via `kapi_get_args(buf, size)`, and exits early with `kapi_exit(status)`.

Linking (`user/user.ld`): everything is linked at **`USER_VA_BASE` = 8 GB** (`. = 0x200000000;`). This is **not PIE**. The RX (`.text`/`.rodata`) and RW (`.data`/`.bss`) sections are aligned on **64 KB** (`-z max-page-size=0x10000`) so that the loader maps them onto distinct pages. **No kernel symbol** is resolved at link time: everything goes through the ABI table.

Compilation flags (`user/Makefile`):

```
-ffreestanding -nostdlib -fno-pic -fno-pie -mgeneral-regs-only \
-O2 -Wall -Wextra -fno-stack-protector -I../kernel/include
```

- `-ffreestanding -nostdlib`: **no libc**. No `printf`, `malloc`, `string.h`… use `applib.h` (§8) and static/local buffers.
- `-mgeneral-regs-only`: integer-only codegen — the **default** for apps. Most apps do **integer / fixed-point arithmetic** (see `tinycalc`, `mandelbrot`).
- **Hardware float is available (opt-in).** The kernel now saves the full FP/SIMD state on every trap (see Kernel internals §6, trap frame), and FP/SIMD is enabled at EL0 (`CPACR_EL1`). To use `float`/`double` in an app, build it **without** `-mgeneral-regs-only` and **with** `-mcpu=cortex-a72` (the Pi 4 core). Basic FP (`+ - * /`, int↔double) compiles to hardware instructions with **no** soft-float runtime; transcendental math (`sin`/`cos`/ `pow`…) lives in newlib's `libm` — see §5.1. For the bare-FP recipe, see the `fptest` target in `user/bin/Makefile` (a target-specific `CFLAGS` override) and the `fptest` proof tool. Opt **only** the FP app in — leave the rest integer-only.
- `-I../kernel/include`: to include `<kern/kapi_abi.h>` (the shared ABI structure).

To add an app, create `user/<name>.c` and **add `<name>.elf` to the `PROGS` variable** of `user/Makefile` (and `<tool>.elf` to the `PROGS` of `user/bin/Makefile` for a tool).

5.1. Apps using the C library (newlib)

The `aarch64-none-elf` toolchain ships **newlib** (`libc` + `libm`). An app can be built against it to use the real `<stdio.h>` (`printf`, `FILE*`, `fopen`/`fseek`), `<stdlib.h>` (`malloc`/`qsort`/`strtod`), `<string.h>`, and `<math.h>` instead of the freestanding helpers (`applib.h`/`umm.h`). This is the foundation for porting large C codebases (e.g. NetSurf).

How it works: `user/libc/onyx_syscalls.c` implements the handful of POSIX stubs newlib bottoms out in (`_sbrk`, `_read`, `_write`, `_open`, `_close`, `_lseek`, `_fstat`, `_gettimeofday`, …) on top of the kapi ABI. `user/libc/crt0libc.S` is the entry point — like `crt0.S` but it calls `exit(main())` so stdio is flushed on the way out. Build flags drop `-ffreestanding`/`-nostdlib` and use `-nostartfiles` (keep our entry, keep `libc`); add `-mcpu=cortex-a72` (FP is required by `printf %f` and `libm`) and link `-lm`. See the `LIBC_PROGS` rule in `user/bin/Makefile` and the proof tool `user/bin/libctest.c`.

Notes / caveats:

- **One allocator.** newlib's `malloc` owns the heap via `_sbrk`→`kapi_sbrk`. Do **not** also link `umm.h` in a newlib app.
- **Files are buffered in RAM.** The kapi file API is sequential (no seek), so `_open` slurps the whole file into memory to give `fseek`/`ftell` full semantics, and a writable file is written back with `kapi_save_file()` on `close`. Fine for resource files; a future `kapi_lseek` would remove the whole-file buffering.
- **Size.** Static newlib pulls a fair amount of code (`printf` float support etc.). Acceptable for big apps; a `nano` variant can be revisited later for small tools.

6. Writing a graphical application

Minimal skeleton (window with kernel-managed widgets):

```
#include "kapi.h"

static void on_ok (unsigned long sender, int ev, long value)
{
    (void) sender; (void) ev; (void) value;
    kapi_widget_set_text (g_label, "cliqué !");
}

static unsigned long g_label;

int main (void)
{
    /* Le canvas est mappé à 12 Go ; fb[y*w + x] = 0x00RRGGBB. */
    unsigned *fb = kapi_create_window (300, 200, "exemple");

    g_label = kapi_add_label  (10, 10, 200, 16, "prêt");
    (void)    kapi_add_button (10, 40, 80, 28, "OK", on_ok);

    kapi_wait_for_exit ();   /* pompe les événements à ~60 fps jusqu'à fermeture */
    return 0;
}
```

Key points:

- `kapi_create_window(w, h, title)` returns a pointer to the **canvas** (pixel buffer of `0x00RRGGBB`, width `w`). The app draws directly into it (no per-pixel call). The variant `kapi_create_window_ex(x, y, w, h, title, flags)` is for explicit placement and `WIN_FLAG_BORDERLESS`.
- **Kernel widgets**: `kapi_add_button/label/checkbox/textbox/progress/slider/textarea/scrollbar_v/scrollbar_h/icon(...)` return an `unsigned long` handle. Manipulate them with `kapi_widget_set_text/get_text`, `get_checked`, `get/set_value`, `set_rect` (move/hide by setting `w=h=0`), `set_icon`.
- **Event loop**: either `kapi_wait_for_exit()` (blocking, simple), or your own loop `while (!kapi_should_exit()) { ...; kapi_pump_events(); kapi_present(); kapi_msleep(16); }` when you animate the canvas yourself.
- **App-drawn UI**: to draw text over your canvas, use `kapi_draw_text(x, y, s, color)` + `kapi_font_width/height()`. Capture the keyboard with `kapi_set_key_handler(fn)` (`GUI_EVENT_KEY` events, `KEY_*`/ASCII values) and the "outside-widget" clicks with `kapi_set_click_handler(fn)` (`GUI_EVENT_CANVAS_CLICK` / `..._MOTION`, coordinates encoded in `value`). Cursor position relative to the window: `kapi_cursor_pos(&x, &y)`.
- **Cooperative**: your app **must yield** regularly (`present`/`msleep`/ `pump_events`/`wait_for_exit`), otherwise it freezes the whole system.

See the demos `demoD.c` (widget gallery), `demoE.c` (textarea + scrollview), and the apps `tinypad.c`, `paint.c`, `mandelbrot.c` for complete examples.

7. Writing a `/bin` tool

A `/bin` tool follows the **same EL1 app model** but reads `stdin`, writes `stdout`, and exits (no window). It is composable via the terminal's pipes.

```c
#include "kapi.h"
#include "applib.h"     /* ax_puts, ax_putln, ax_strlen, ax_itoa, ... */

int main (void)
{
    char args[128];
    kapi_get_args (args, sizeof args);   /* la ligne d'arguments (chaîne) */

    /* Lire stdin et le réécrire en majuscules, par exemple : */
    char buf[256];
    int n;
    while ((n = kapi_stdin_read (buf, sizeof buf)) > 0)
    {
        for (int i = 0; i < n; i++)
            if (buf[i] >= 'a' && buf[i] <= 'z') buf[i] -= 32;
        kapi_stdout_write (buf, (unsigned) n);
    }
    return 0;   /* code de sortie récupérable via kapi_wait dans l'appelant */
}
```

- `kapi_get_args(buf, size)`: the entire argument line as **a single string** separated by spaces (there is no `argv` array; parse the first word yourself, etc.).
- `kapi_stdin_read(buf, n)`: reads the task's stdin (`0` = EOF). `kapi_stdout_write`: writes stdout.
- To read a file passed as an argument: `kapi_open/read/close`. To list a directory: `kapi_opendir/readdir/closedir`.

The existing tools to study: `ls`, `cat`, `grep`, `wc`, `echo`, `page`, `rm`, `mkdir`, `touch`, `cp`, `mv`, `ps`, `kill`, `run`, `keyb` (in `user/bin/`).

8. The `applib.h` library

`user/applib.h` is **header-only** (no libc). It provides:

- **Strings**: `ax_strlen`, `ax_streq`, `ax_strcat(dst, cap, &pos, src)` (concat without overflow), `ax_app_path(dst, cap, name, suffix)` (builds `SD:apps/<name><suffix>`), `ax_itoa(v, buf)`, `ax_fmt2(d, v)` (2-digit decimal).
- **Console**: `ax_puts(s)`, `ax_putln(s)` (to stdout).
- A minimal **`.ini` reader**: `app_ini_load("config.ini")` (from the app's folder via `kapi_app_dir`) or `app_ini_load_path("SD:skins/theme.txt")`; then `app_ini_get(section, key, default)` and `app_ini_get_int(...)`. Sections `[xxx]`, lines `key=value`, comments `;`/`#`.
- **App-drawn widgets** (not kernel widgets): `ax_dropdown` (drop-down list) and `ax_colorpick` (palette-based color picker), with `*_draw(...)` and `*_click(...)`. The app draws them in its canvas and routes the clicks via its `set_click_handler`. Used by `theme.c` and `mandelbrot.c`.

There is also `user/httpc.h` — a header-only **HTTP/1.0 client** over the v21 TCP socket calls. It does **no allocation** (the caller passes the response buffer, so all memory stays in the app's address space): `http_get(url, buf, cap, &resp)`, `http_post(...)`, or the general `http_request(method, url, headers, body, len, buf, cap, &resp)`. Plain HTTP only (no TLS). Used by `/bin/wget`. This is the recommended pattern for application-level protocols: build them in a user library on top of the kernel's transport kapis, rather than adding them to the ABI.

For **REST / web-API** clients there is a reusable C++ class, `user/http.hpp` (`HttpClient`/`HttpResponse`) — a header-only, freestanding **HTTP/1.1** client that works in any app (integer-only or newlib). It adds, over `httpc.h`: custom default headers (chainable `.bearer(token)`, `.accept(type)`, `.header(name,value)`), JSON helpers (`post_json`/`put_json`), response-header lookup (`r.header("Content-Type", out, cap)`), and **chunked** transfer decoding. Same no-allocation model (the caller passes `char buf[N]`; the body points into it, NUL-terminated). Errors are a negative `r.status` (`HttpError`). Example: `HttpClient api; api.bearer(tok).accept("application/json"); auto r = api.get(url, buf, sizeof buf); if (r.ok()) ….` See `/bin/httpget` for a working demo. **HTTPS:** the class has a transport seam (`http://` = plain kapi TCP; `https://` = TLS). TLS is provided by `user/tls/onyx_tls.hpp` — **mbedTLS ≥3.6.3** over the kapi sockets, with a **buffered** BIO (Circle's `CSocket::Receive` discards the remainder of a TCP segment on a short read, so we read whole segments) and **software-only crypto** (the Pi 4's Cortex-A72 has no ARMv8 crypto extensions). **Verified end-to-end on real hardware** — `httpsget` downloads real pages. Same model NetSurf uses (HTTPS from an external stack, libcurl+OpenSSL). To enable it in a **newlib** app: `#define ONYX_HTTP_TLS` before `#include "http.hpp"` and link the cross-built mbedTLS libs — `make -C user/tls` then `make -C user/bin MBEDTLS_DIR=../tls/mbedtls` (see `user/tls/README.md`). The freestanding default (no `ONYX_HTTP_TLS`) keeps `http://` only and returns `HTTP_ERR_HTTPS` for `https://`. Demo: `/bin/httpsget`. **Not yet secure:** the RNG is a software PRNG (not the HW RNG, which stalls on the Pi 4) and certificate verification is OFF — see the TLS README.

For **images** there is a reusable decoder, `user/img/image.hpp` (`onyximg::decode(data, len, &w, &h)`) — built on the cross-compiled **zlib + libpng + libjpeg**. It sniffs the format (PNG signature / JPEG SOI) and decodes a byte buffer into a freshly `malloc`'d array of `0xAARRGGBB` pixels (8-bit alpha in the top byte, which the canvas ignores when blitting). Same split as TLS: the libraries are cross-built once (`make -C user/img`, sources pinned in `user/img/README.md` — zlib 1.3.1, libpng 1.6.44, libjpeg IJG v9f), the Onyx glue is header-only. It is a **newlib** component (uses `malloc` + the libs), so it is OPT-IN: `make -C user/bin`

`IMG_DIR=../img` builds the `/bin/imgtest` demo (decodes an embedded PNG and prints its size). This is the same model NetSurf uses — link libpng/libjpeg/zlib, decode behind one wrapper. Note: `image.hpp` is for full-colour web images; keep `user/bmp.hpp` for the magenta-keyed `0x00RRGGBB` icons loaded from SD.

For a **NetSurf-style framebuffer GUI** there is an Onyx **libnsfb** surface backend, `user/nsfb/onyx_surface.c`. libnsfb is NetSurf's framebuffer abstraction (its software plotters draw into a surface buffer); this backend makes an Onyx window **content canvas** that surface: `initialise` → `kapi_create_window` (the `0x00RRGGBB` canvas *is* the framebuffer), `update` → `kapi_present`, and `input` bridges Onyx's callback-driven pointer/key events (`kapi_set_pointer/key_handler` + `pump_events`) into libnsfb's poll-style event queue. The format is `NSFB_FMT_XRGB8888` — on little-endian the plotter packs `0x00RRGGBB`, exactly the canvas layout (no R/B swap). The vendored libnsfb is **unpatched**: the backend registers under the name `"onyx"` (resolved with `nsfb_type_from_name("onyx")`) via a constructor that Onyx's `crt0` runs from `.init_array`. Cross-built once (`make -C user/nsfb`, pinned in `user/nsfb/README.md`), then OPT-IN: `make -C user/bin NSFB_DIR=../nsfb` builds the `/bin/nsfbdemo` demo (draws shapes through libnsfb and tracks the cursor). `libnsfb.a` is linked `--whole-archive` so the surface's registration constructor is not dropped.

The **NetSurf core library stack** also cross-builds for Onyx — `user/netsurf/` builds libwapcaplet, libparserutils, libnsutils, libnsgif, libnsbmp, libhubbub (HTML), libcss (CSS) and libdom (DOM) against newlib (`make -C user/netsurf`, versions pinned in `user/netsurf/README.md`). The whole stack **links clean** — no undefined symbols — against `crt0libc` + `onyx_syscalls` + newlib. Three Onyx-side fixes made it self-contained: libparserutils is built `-DWITHOUT_ICONV_FILTER` (use its own charset codecs; newlib has no `iconv`), everything is built `-fcommon` (the code predates GCC 10's `-fno-common`), and `pread`/`pwrite` were added to `onyx_syscalls.c`. Code that the upstream buildsystem normally generates with host tools is reproduced in the Makefile: perl for the libparserutils charset aliases and libhubbub entities, a host-compiled `gen_parser` for the libcss property parsers, and a gperf-free element-type table for libhubbub (`user/netsurf/gen/`). This is NetSurf brick 7. Brick 8 — an Onyx **fetch scheme handler** (`user/netsurf/onyx_fetch.c`) — drives the NetSurf fetch API over the Onyx TCP kapis (+ gzip via zlib), the curl-fetcher's role without libcurl. Brick 9 wires the whole **NetSurf core + its framebuffer frontend** to the `user/nsfb` `"onyx"` libnsfb surface + the `user/img` decoders: `make -f user/netsurf/netsurf-app.mk stage` builds `netsurf.elf` (182 TUs + the libs) and installs it as the `NetSurf` desktop app. **It runs on Onyx** — the window opens and real web pages render through the full HTML/CSS engine (currently slow; the `onyx_main.c` entry shim passes `-f onyx` to select the window surface). A console `nstest` (`netsurf-app.mk nstest`) smoke- tests each library brick. See `user/netsurf/README.md`.

And `user/uikit.h` — a **retained-mode widget toolkit** drawn entirely in the app's canvas, driven by the kernel's **pointer stream** (ABI v22: `set_pointer_handler` → `GUI_EVENT_PTR_MOVE/DOWN/UP/ENTER/LEAVE` with client coords). Same memory model as the rest: widgets live in a caller-provided **fixed pool** (`ui_widget pool[N]` in the app's `.bss`, freed automatically on exit — no user `malloc`, no kernel object behind a widget). `ui_init`, `ui_button`/`ui_label`/ `ui_checkbox`/`ui_textbox`, `ui_on_event` (fed from the app's pointer + key handlers), `ui_draw`. The **textbox** is a single-line editor: caret, Backspace/Delete, arrows, Home/End, `Tab` to move focus, an optional password mask (`ui_set_password`), read with `ui_get_text`. `tinycalc` (buttons) and `wpaconf` (a form of textboxes) use the toolkit. This is the forward path for widgets: new ones are added here, in userland, with no kernel/ABI change. The older **kernel-drawn widgets** (`add_button`..., §earlier) still work and coexist; apps choose one model per window.

Dynamic memory + C++ apps

Apps have **no `malloc` by default** — they use static buffers + the stack (both in the app's address space). For dynamic memory, include `user/umm.h`: a small user allocator (size-class free lists + a `kapi_sbrk` arena).

`umm_malloc` / `umm_free` / `umm_calloc` / `umm_realloc`. The heap lives at `USER_HEAP_BASE` (10 GB); its pages are owned by the address space, so they are **freed automatically when the app exits** and show up in the app's page count (`ps` / `memmon`). `kapi_sbrk` is the underlying primitive (rarely called directly). `/bin/heaptest` exercises it.

C++ apps are supported (freestanding subset — no exceptions, no RTTI, no STL):

- Name the source `*.cpp`; the user `Makefile` builds it with `g++` (`-fno-exceptions -fno-rtti -fno-threadsafe-statics -fno-use-cxa-atexit`).
- Include `user/onyxpp.hpp`: it defines `operator new`/`delete` (on `umm`) and the runtime stubs (`__cxa_pure_virtual`, `__dso_handle`, `__cxa_atexit`/`atexit` no-ops). Global constructors run via `crt0.S` (the `.init_array` walk); static destructors are **not** run (the app exits and its address space is reclaimed).
- Classes, inheritance and virtual methods (vtables) work; `new`/`delete` go through the user heap. No `std::string`/`std::vector` — write small containers on `umm` as needed. See `user/cppdemo.cpp` for a working example.

9. Packaging an app: `.app`, icons, `config.ini`

Layout of an application on the card (produced by `make stage`):

```
SD:apps/<nom>.app/
  main.elf       l'ELF de l'app (obligatoire)
  icon.bmp       icône 40×40, BMP 24 bpp ; le magenta 0xFF00FF est transparent (optionnel)
  app.txt        métadonnées (display name + catégorie) lues par le launcher (optionnel)
  config.ini     configuration de l'app, lue via app_ini_load() (optionnel)
```

The **app name** is the base name of the `.app` folder (without the suffix). That is what you put in `/etc/autostart` / `/etc/quicklaunch.txt` and what `kapi_list_apps` returns.

`app.txt` — friendly metadata for launchers (`key = value`, no section, read with `app_ini_load_path` / `app_ini_get(0, …)`). Every app under `SD:apps/` ships one:

```
name     = Text Editor          ; display name shown under the icon
category = Productivity          ; Games, Graphics, Productivity, Internet, System, Demos, Shell
```

The current app-drawer (`applist`) still labels icons by folder name; `app.txt` is the groundwork for a **category-grouping launcher** (groups icons by `category`, shows `name`). The two shell components `panel`/`applist` carry `category = Shell` so a launcher can exclude them.

Icons — `tools/gen_assets.py` procedurally generates the 40×40 BMPs (BGR bottom-up, 4-byte padding) for all the apps (a document for `tinypad`, a calculator for `tinycalc`, a glider for `life`, etc.) and the "9 squares" glyph of the *apps* button in the panel. `tools/preview_icons.py` produces a PNG preview montage. Workflow: run `gen_assets.py` (writes the `icon.bmp` files into `sdcard/apps/<name>.app/`), then `preview_icons.py` to check visually.

`config.ini` — example read by `inidemo` / `voronoy` / `panel`:

```
; commentaire
greeting = bonjour
[app]
name = Mon App
version = 1
```

```
[display]
barwidth = 40
```

Screenshots and documentation

- **Screenshots (simulated but faithful)**: `tools/screenshot/render.py` loads the **real** skins (`SD:skins/wings/button/closebgs.bmp`), the **real** font `circle/lib/font8x16.cpp` and the icons `SD:apps/<name>.app/icon.bmp`, and reproduces the drawing routine of each app — so the output matches the real OS. Run `python tools/screenshot/render.py` → writes the PNGs into `screenshots/` (`desktop.png` = overview, plus one screenshot per app, window only on a transparent background). **To add an app**: write `app_<name>()` that reproduces the `redraw()` of `user/<name>.c` (same colors/coords), add `("<name>", app_<name>)` to the `APPS` list, and re-run.
- **Word/PDF exports**: `docs/build_docs.py` converts each `.md` in `docs/` into `.docx` (via `pandoc`) then into `.pdf` (via Word/`docx2pdf`), in `docs/exports/`. Run `python docs/build_docs.py` after any modification to the `.md` files. Prerequisites (once): `pip install python-docx docx2pdf pypandoc_binary`.
- **Reminder of the project rule** (see `CLAUDE.md`): any modification of a `kapi` function or of an app **updates the docs in the same go**, regenerates the affected screenshot if the visual changes, then regenerates the exports.

10. Extending the `kapi` ABI

The ABI is **append-only**. To add a kernel function callable by apps, there are **5 points** to touch (all in the same direction, at the end):

1. **`kernel/include/kern/kapi_abi.h`** — add the function pointer **at the end** of `struct TKApiTable` (never in the middle, never reorder), with a version comment, and **increment** **`KAPI_ABI_VERSION`**.

   ```
   // --- v22 additions ---   (next free version; v21 added the TCP sockets)
   int (*ma_fonction) (int arg);
   ```

2. **`kernel/sys/kapi.cpp`** — implement `extern "C" int kapi_ma_fonction(int arg)`. It runs in the app's context (use `CurrentAS()` for the current address space if needed).

3. **`kernel/sys/kapitable.cpp`** — declare the `extern "C"` prototype and **assign the pointer** in `KApiTableInit()`: `t->ma_fonction = kapi_ma_fonction;`.

4. **`user/kapi.h`** — add the inline wrapper:

   ```
   static inline int kapi_ma_fonction (int arg) { return KT->ma_fonction (arg); }
   ```

5. Rebuild (`make`). The apps that want the new function use it; the old ones keep working (they ignore the new field).

> **Golden rule:** never change the signature or the order of an existing field. If some semantics must change, add a **new** entry. An app can query `((const struct TKApiTable *)KAPI_TABLE_VA)->version` to find out what is available.

If you add a new **GUI event** or a **window flag**, keep the values synchronized between `kernel/gui/window.h` and the `#define`s in `user/kapi.h` (commented "must match").

11. Coding conventions

- **Kernel (C++)**: Circle style. `CXxx` classes, `m_Xxx` members, CamelCase methods, `boolean`/`TRUE`/`FALSE` and Circle's `u8/u16/u32/u64` types. No exceptions or RTTI. `new`/`delete` go through Circle's heap.
- **Userland (C)**: freestanding C. `ax_` prefix for the `applib.h` helpers. Globals `g_xxx`. Bounded static buffers (no dynamic allocation on the app side in general).
- **kapi**: `extern "C"` functions named `kapi_xxx` on the kernel side; inline wrappers `kapi_xxx` on the app side.
- Respect the **comment density** and the **idiom** of the file you are modifying.
- **Git**: commit into the **Onyx repo** explicitly — the current working directory (cwd) drifts; a bare `git` may land in the wrong repo. (Note: `circle/` is not committed in this repo.)

12. Debugging on hardware

Bring-up is done **directly on the Pi 4** (no QEMU raspi4b). Tools:

- **On-screen exception dump**: an EL1 synchronous fault (or an EL0 fault) paints a panic + register dump on the HDMI framebuffer (`PanicToScreen` + Circle's handler). Note the `ELR` (faulting PC).
- **`addr2line`**: `aarch64-none-elf-addr2line -e kernel8-rpi4.elf <ELR>` to locate the faulting line (keep the unstripped `.elf` next to the `.img`).
- **Serial console**: `config.txt` must have `enable_uart=1` (PL011 clock). The boot log goes **also** to the HDMI screen (`CScreenDevice`) so it is readable without a serial cable.
- **Post-mortem console**: on an app's exit, the compositor clears and the logger is shown on the framebuffer (see Kernel internals §11).

13. Known pitfalls

- **Hardware float is opt-in.** Apps are integer-only by default (`-mgeneral-regs-only`); the kernel now saves the full FP/SIMD state on every trap, so an app may opt into `float`/`double` by building without `-mgeneral-regs-only` and with `-mcpu=cortex-a72` (see §5). Caveat: if **preemptive** scheduling is ever enabled (today the scheduler is purely cooperative), it **must** switch via the full trap frame — the FP save lives there, not in the cooperative `TaskSwitch`.
- **L3 tables shared with the kernel.** On the kernel side, never free an L3 table from the user area without checking that it is not shared with the kernel's L2 (cf. Kernel internals §4). Otherwise: global corruption.
- **Do not free the ABI table page.** It is global to the kernel; the destruction of an address space already skips it.
- **`DEPTH=32` for Circle.** `GImage` renders 32-bit; forgetting `-d DEPTH=32` (or changing `DEPTH` without `make clean` in `circle/lib`) gives wrong colors/breakage.
- **Cooperative.** Any app loop without `present`/`msleep`/`yield` freezes the system (no preemption).
- **Circle LF renormalization.** On Windows, Circle is checked out in CRLF; renormalize once (cf. §2) otherwise the build breaks.
- **The right Circle.** Patch `Zircon/circle`, not another clone.